POWER BI · LESSON 8 OF 8

Databricks: Notebooks, Delta Lake, Spark Jobs and Unity Catalog

Work on a lakehouse with Databricks. Write notebooks in SQL and PySpark, store tables as Delta Lake with merges and time travel, run Spark jobs on a schedule, and govern data with Unity Catalog so that reports read trusted tables.

Gap-fill lesson | ✦ Databricks 2026 | Advanced | ⏱ 150 min

By the end of this lesson you can

- Explain the Databricks workspace, clusters and notebooks
- Write a Delta table with SQL and PySpark, and explain why Delta adds reliability
- Use MERGE, time travel and schema checks on Delta tables
- Schedule a Spark job and understand how it differs from a local pandas script
- Organise data with Unity Catalog catalogs, schemas and permissions

Datasets used

`adventure_cycles/fact_sales.csv` `adventure_cycles/dim_customer.csv`

What Databricks adds

Databricks is a data platform built on Apache Spark. Spark processes data in parallel across a cluster of machines, so it handles data too large for one computer. Databricks adds managed clusters, notebooks, scheduled jobs and a governance layer called Unity Catalog. Its storage format, **Delta Lake**, adds reliable updates and history to files stored in cloud object storage.

For a Power BI analyst the practical point is simple: many companies keep their reporting tables in Databricks, and reports connect to them through the SQL warehouse endpoint or through Power BI's Databricks connector.

CASE STUDY **Millions of rows, one daily refresh**

The sales file has grown to 80 million rows and no longer fits in a Power BI import. The data team keeps the detail in Delta tables in Databricks and builds a daily summary table with a scheduled Spark job. The report reads the summary, and the detail is queried only when someone drills down.

Workspace, clusters and notebooks

- **Workspace**: where notebooks, jobs and tables are organised.
- **Cluster** (or serverless compute): the machines that run your code. Clusters start, scale and stop; you pay while they run, so auto-termination matters.
- **Notebook**: a document of cells that mix Markdown, SQL, Python and PySpark. Each notebook attaches to a cluster and runs one cell at a time.

A notebook can switch language per cell with a magic command such as `%sql`, so one notebook can read a table in SQL and transform it in PySpark.

Reading data with PySpark

PySpark is the Python API for Spark. A DataFrame in Spark looks like a pandas DataFrame, but the work is split across the cluster and runs lazily: transformations are planned until a result is needed.

Python

```python
# In a Databricks notebook, `spark` is already created
sales = spark.read.table("retail.bronze.sales_raw")

summary = (
    sales
    .filter("order_date ≥ '2025-01-01'")
    .groupBy("region", "category")
    .agg({"sales": "sum", "profit": "sum"})
    .withColumnRenamed("sum(sales)", "total_sales")
    .withColumnRenamed("sum(profit)", "total_profit")
)

summary.write.mode("overwrite").saveAsTable("retail.gold.sales_summary_2025")
```

The names `retail.bronze.sales_raw` and `retail.gold.sales_summary_2025` follow the bronze, silver and gold layering: raw data, cleaned data and reporting data. This is the common pattern for lakehouse work.

::::note Lazy evaluation Calling `filter` or `groupBy` does not run anything. Spark runs the plan when you call an action such as `write`, `count` or `show`. A slow notebook is often a plan that recomputes data many times, which `cache()` or writing an intermediate table can fix. ::::

Delta Lake: reliable tables on files

A Delta table is a set of Parquet data files plus a transaction log that records every change. The log gives you:

- **ACID writes**: an update either completes or does not happen, so readers never see half-written data.
- **MERGE**: insert new rows and update changed ones in one statement, which is the upsert from the SQL incremental-loading lesson.
- **Time travel**: query the table as it was at an earlier version or time.
- **Schema enforcement**: writes with the wrong columns or types are rejected.

SQL

```sql
-- Upsert new and corrected order lines into the Delta table
MERGE INTO retail.silver.fact_sales AS target
USING retail.bronze.sales_today AS source
ON target.line_id = source.line_id
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *;

-- What did the table look like yesterday?
SELECT COUNT(*) AS lines, SUM(sales) AS total_sales
FROM retail.silver.fact_sales VERSION AS OF 12;

-- The history of changes
DESCRIBE HISTORY retail.silver.fact_sales;
```

Time travel is valuable for audits and for undoing a bad load: restore the previous version rather than rebuilding the table. Check retention settings first, because old versions are removed after a period.

::::warn MERGE needs a unique key If the source has two rows with the same `line_id`, MERGE fails or updates the same target row twice. Deduplicate the source before the merge, using the latest record for each key. ::::

Scheduled Spark jobs

A notebook becomes production work when it runs as a **job**:

1. Open the notebook, choose **Schedule** or create a job, and select a cluster or serverless compute.
2. Set the schedule (for example, every day at 5 a.m.) and the parameters.
3. Add a notification for failure, and set a retry for transient errors.
4. Check the run history after the first few runs, and compare the output totals with the source.

A job should be idempotent, like the loads in the SQL lesson: running it twice should not double the totals. Overwrite a summary table, or merge into a fact table, rather than appending.

Unity Catalog: governance across the platform

Unity Catalog is the central place to organise and secure data. Its three-level name is **catalog.schema.table**:

- **Catalog**: a top-level container, often one per environment or business unit (for example `retail`).
- **Schema**: a group of related tables (for example `gold` for reporting tables).
- **Table or view**: the data itself.

Permissions are granted on these objects, so a group can read `retail.gold` but not `retail.bronze`:

```sql
GRANT USE CATALOG ON CATALOG retail TO `analysts`;
GRANT USE SCHEMA ON SCHEMA retail.gold TO `analysts`;
GRANT SELECT ON SCHEMA retail.gold TO `analysts`;
```

Unity Catalog also records lineage, which shows which tables and notebooks produced a table. Use it to answer "where does this number come from?" before a report is trusted.

::::tip Give reports gold tables only Point Power BI at the gold layer, not at raw bronze tables. Gold tables are the cleaned, agreed definitions, so the report does not repeat cleaning logic and does not expose raw data to viewers. ::::

Connecting Databricks to Power BI

Power BI connects to a Databricks SQL warehouse through its Databricks connector, using the server hostname, HTTP path and an authentication method approved by the data team. Import mode suits daily summaries; DirectQuery suits large tables that must stay current. Choose the mode the same way as in the modes lesson: by data size, freshness and the number of users.

Comparing the platforms

Need	Fabric	Databricks
Storage	OneLake	Cloud storage with Delta tables
Large-scale code	Spark notebooks in a lakehouse	Spark notebooks and jobs
Governance	Workspaces, endorsement, lineage	Unity Catalog
Reporting	Native semantic models	Connect Power BI to a SQL warehouse

Both platforms run Spark, store tables in open formats, and feed Power BI. Learn the concepts once; the names change.

::::key What to remember - Databricks runs Spark on managed clusters; notebooks mix SQL and PySpark. - Delta Lake adds ACID writes, MERGE, time travel and schema checks to files. - Build summary tables as scheduled jobs, and make jobs safe to re-run. - Unity Catalog governs catalogs, schemas and tables with permissions and lineage. - Point reports at gold tables that hold agreed definitions. ::::

Quick quiz

5 questions · tap an answer to check it

1. What is Delta Lake?

A A type of chart

B A table format that adds transaction logs, MERGE and time travel to files

C A Power BI gateway

D A programming language

Answer: B. Delta stores Parquet files with a log of every change.

2. What does time travel let you do?

A Make the cluster faster

B Query a table as it was at an earlier version

C Rename columns

D Skip a refresh

Answer: B. Time travel reads earlier table versions for audits and recovery.

3. Why must a MERGE source have unique keys?

A Keys are optional in Delta

B Duplicate keys in the source can make the merge fail or update one target row twice

C MERGE only works on text

D It speeds up the cluster

Answer: B. Deduplicate the source before merging.

4. In `retail.gold.sales_summary`, what is `gold`?

A The catalog

B The schema

C The table

D The cluster

Answer: B. The three-level name is catalog.schema.table.

5. Why should Power BI read gold tables rather than bronze?

A Bronze tables are faster to read

B Gold tables hold the cleaned, agreed definitions and avoid exposing raw data

C Gold tables cannot be queried

D Bronze tables are always deleted

Answer: B. Gold is the reporting layer.

EXERCISE **Design a daily job**

A retail team loads raw sales files each night into a bronze table. Design the daily job that produces a gold summary of sales by region and month for Power BI. Write the steps, state whether the summary is overwritten or merged and why, list one check to run before publishing, and name the Unity Catalog permission that lets the analysts read the summary but not the raw data.

Show solution (worked answer)

Steps: read the bronze table, deduplicate by line ID, filter the new period, aggregate sales and profit by region and month, write to the gold schema. Overwrite the summary because it is derived in full each run (re-runs give the same result). Check that the total sales in the summary equal the total in the cleaned silver table for the same period. Permission: `GRANT SELECT ON SCHEMA retail.gold TO analysts` with no grant on the bronze schema.

Practise more online

Free, trusted places to test yourself on this topic. Links open the original site in a new tab.